Ordenação II

Estruturas de Dados



UnB | CIC

Definição

Ordenação é o ato de colocar os elementos de uma sequência de dados em uma ordem predefinida.

Definição

Ordenação é o ato de colocar os elementos de uma sequência de dados em uma ordem predefinida.

A eficiência de um algoritmo de ordenação está relacionada ao número de itens que estão sendo processados.

Definição

Ordenação é o ato de colocar os elementos de uma sequência de dados em uma ordem predefinida.

A eficiência de um algoritmo de ordenação está relacionada ao número de itens que estão sendo processados.

O número total de comparações será a maneira mais comum de medir um procedimento de ordenação.

Quick Sort

- É um algoritmo eficiente de ordenação por divisão e conquista.
- Baseado em particionamento: Particionar significa escolher um elemento qualquer, chamado de *pivot*, e colocá-lo em uma posição tal que todos os elementos à esquerda são menores ou iguais e todos os elementos à direita são maiores.

Quick Sort

1. Escolher um pivô:

- Pode ser o primeiro elemento, o último, um aleatório ou a mediana.

2. Particionar os elementos:

- Elementos menores que o pivô ficam à esquerda.
- Elementos maiores que o pivô ficam à direita.

3. Recursivamente ordenar:

- Aplicar Quicksort nas partições esquerda e direita.

Quick Sort

Array inicial: [3, 6, 8, 10, 1, 2, 1]

- **Escolher pivô:** 6

Particionamento:

- Esquerda: [3, 1, 2, 1]
- Meio: [6]
- Direita: [8, 10]

Quick Sort

Aplicar Quicksort na esquerda:

- Array: [3, 1, 2, 1]
- **Pivô:** 3
- Particionamento:
  - Esquerda: [1, 2, 1]
  - Meio: [3]
  - Direita: []

Quick Sort

Combinar resultados da esquerda:

- [1, 1] + [2] + [3] = [1, 1, 2, 3]

Quick Sort

Aplicar Quicksort na direita:

- Array: [8, 10]
- **Pivô:** 8
- Particionamento:
  - Esquerda: []
  - Meio: [8]
  - Direita: [10]

Quick Sort

Combinar resultados da direita:

- [] + [8] + [10] = [8, 10]

Quick Sort

Combinação final:

- [1, 1, 2, 3] + [6] + [8, 10] = **[1, 1, 2, 3, 6, 8, 10]**

Quick Sort

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)
```

Quick Sort

Vantagens:

- Melhor caso: O(n log n)
- Rápido para grandes volumes de dados.
- Boa performance prática, mesmo com entrada desordenada.

Desvantagens:

- Pior caso $O(n^2)$ (pode ser mitigado com escolhas melhores de pivô).
- Não é estável (não preserva a ordem de elementos iguais).

Quick Sort

Melhor Caso: O melhor caso do Quicksort acontece quando o pivô divide o array em duas partições aproximadamente iguais a cada iteração. Isso resulta em uma árvore de recursão balanceada.

- A cada passo, o array é dividido pela metade, reduzindo o tamanho de forma exponencial.
- Para um array de tamanho n, temos no máximo log(n) níveis de recursão.
- Cada nível realiza O(n) operações para particionar os elementos.
- Complexidade do melhor caso: O(n log n)

Quick Sort

Pior Caso: O pior caso do Quicksort acontece quando o pivô divide o array de forma desequilibrada, isto é, uma partição contém todos os elementos, e a outra está vazia ou tem apenas um elemento.

- A cada passo, o tamanho do array é reduzido em apenas 1 elemento.
- Isso resulta em uma árvore de recursão desequilibrada, com n níveis.
- Cada nível realiza O(n) operações para particionar os elementos.
- Complexidade do pior caso: $O(n^2)$

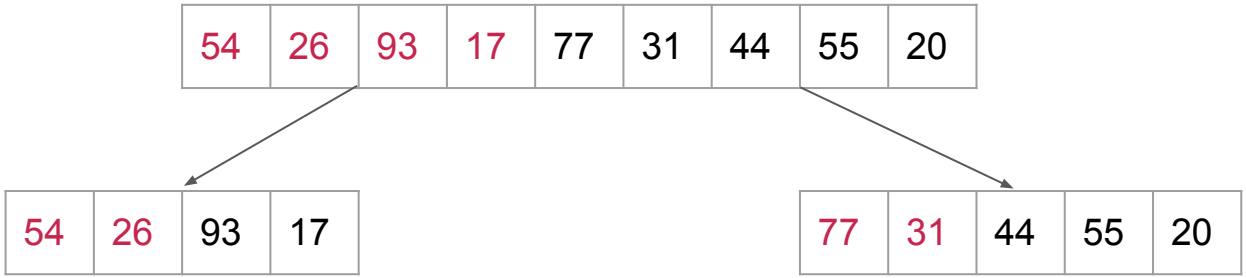
Merge Sort

- O merge sort, ou ordenação por mistura, é um exemplo de algoritmo de ordenação por comparação do tipo dividir-para-conquistar.
- Sua ideia básica consiste em Dividir e Conquistar.

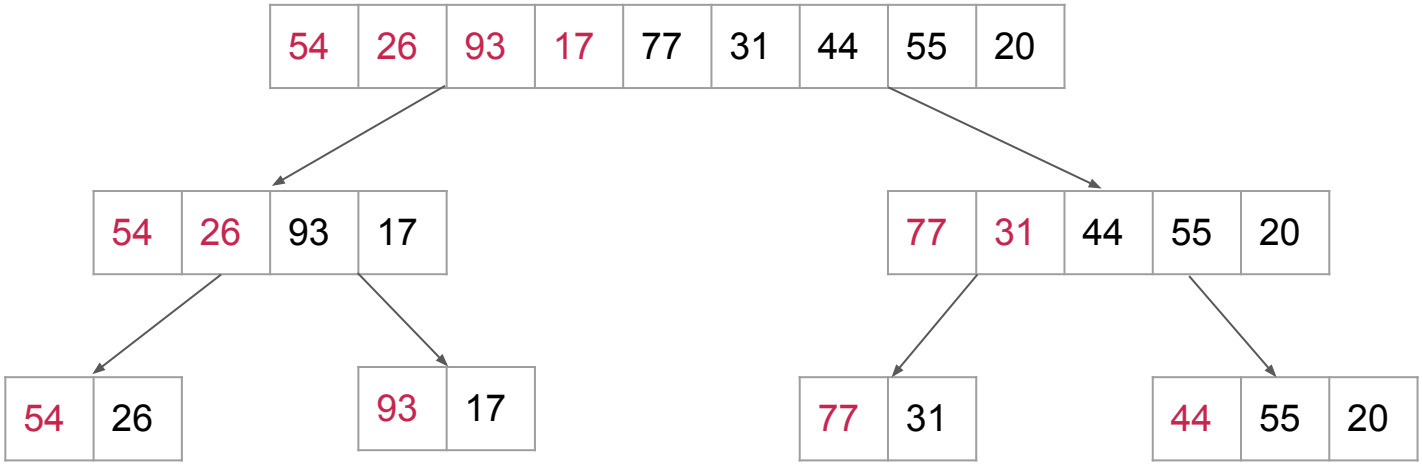
Merge Sort

54	26	93	17	77	31	44	55	20

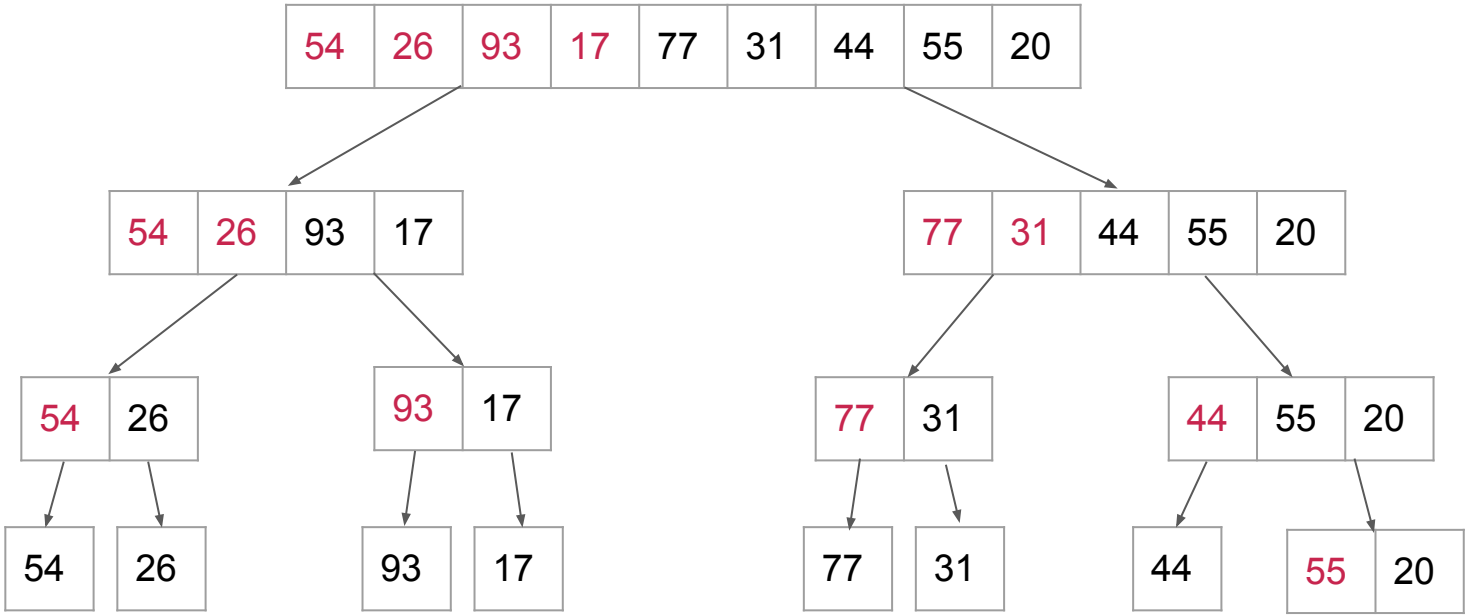
Merge Sort



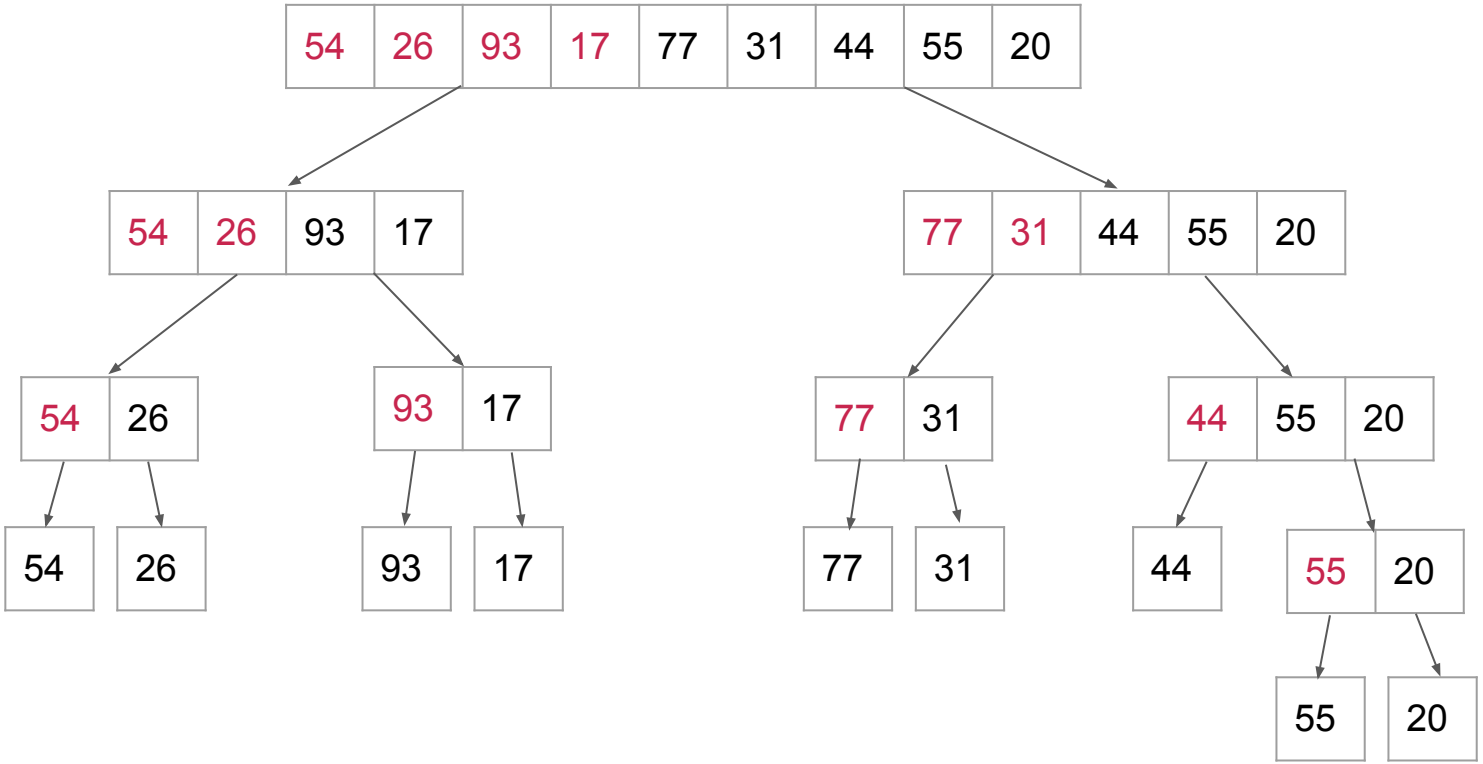
Merge Sort


54 26 93 17 77 31 44 55 20
54 26 93 17
77 31 44 55 20
54 26
93 17
77 31
44 55 20

Merge Sort


54 26 93 17 77 31 44 55 20
54 26 93 17
77 31 44 55 20
54 26
93 17
77 31
44 55 20
54
26
93
17
77
31
44
55 20

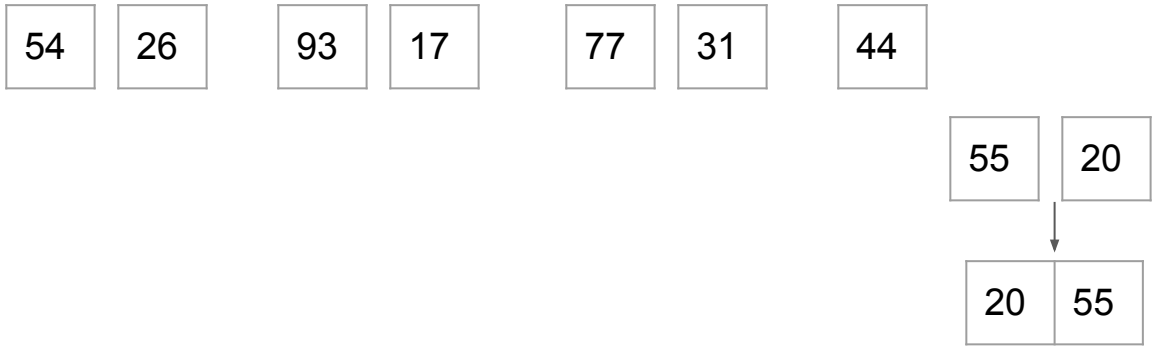
Merge Sort


54 26 93 17 77 31 44 55 20
54 26 93 17
77 31 44 55 20
54 26
93 17
77 31
44 55 20
54
26
93
17
77
31
44
55 20
55
20

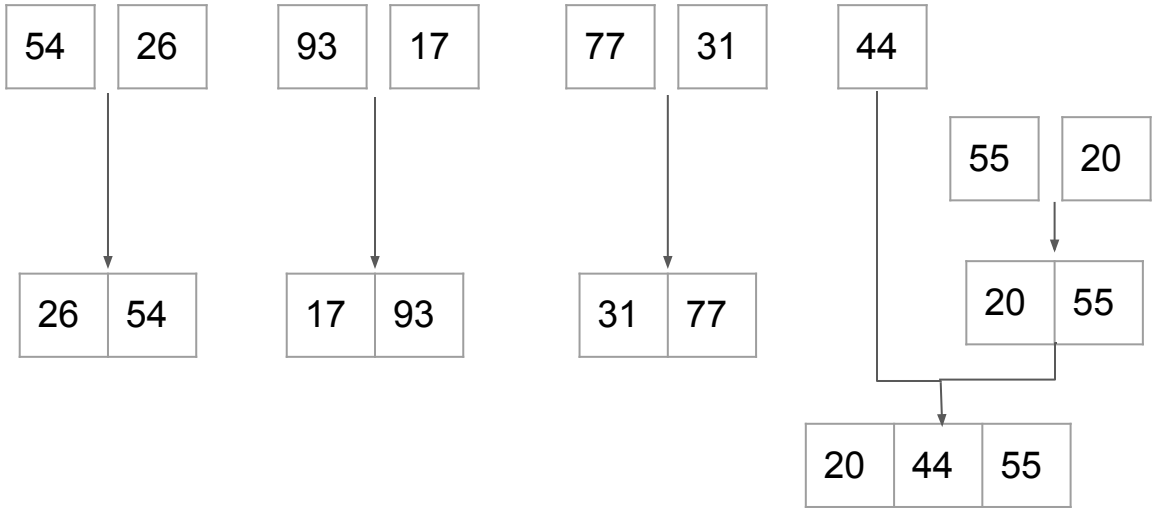
Merge Sort



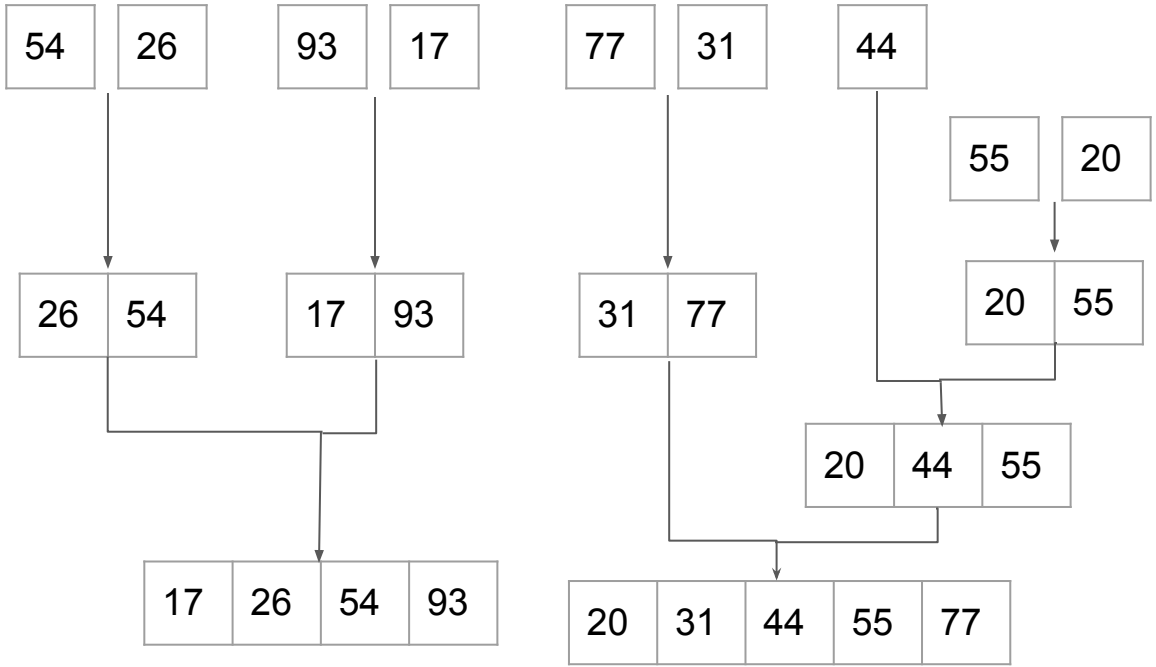
Merge Sort



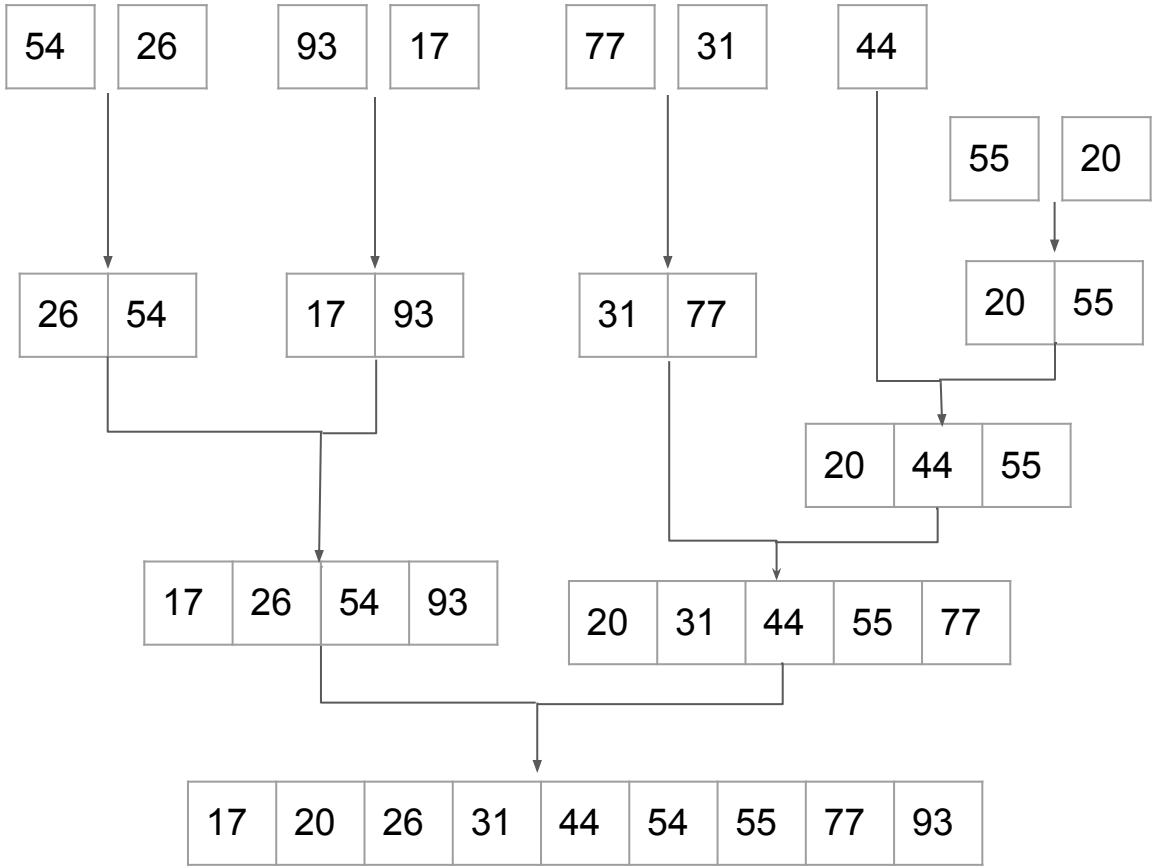
Merge Sort



Merge Sort


54
26
93
17
77
31
44
55
20
26
54
17
93
31
77
20
55
20
44
55
17
26
54
93
20
31
44
55
77

Merge Sort


54 26 93 17 77 31 44 55 20
26 54 17 93 31 77 20 55
20 44 55
17 26 54 93
20 31 44 55 77
17 20 26 31 44 54 55 77 93

Merge Sort

```
def merge_sort(vetor):
    if len(vetor) < 2:
        return vetor
    meio = len(vetor) // 2
    esquerda = merge_sort(vetor[:meio])
    direita = merge_sort(vetor[meio:])
    return merge(esquerda, direita)
```

Merge Sort

```
def merge_sort(vetor):
    if len(vetor) < 2:
        return vetor
    meio = len(vetor) // 2
    esquerda = merge_sort(vetor[:meio])
    direita = merge_sort(vetor[meio:])
    return merge(esquerda, direita)
```

```
def merge(esquerda, direita):
    vetor = []

    while esquerda and direita:
        if esquerda[0] <= direita[0]:
            vetor.append(esquerda.pop(0))
        else:
            vetor.append(direita.pop(0))

    if esquerda:
        vetor.extend(esquerda)
    if direita:
        vetor.extend(direita)

    return vetor
```

Merge Sort

```
def merge_sort(vetor):
    if len(vetor) < 2:
        return vetor
    meio = len(vetor) // 2
    esquerda = merge_sort(vetor[:meio])
    direita = merge_sort(vetor[meio:])
    return merge(esquerda, direita)
```

$$O(n \log n)$$

```
def merge(esquerda, direita):
    vetor = []

    while esquerda and direita:
        if esquerda[0] <= direita[0]:
            vetor.append(esquerda.pop(0))
        else:
            vetor.append(direita.pop(0))

    if esquerda:
        vetor.extend(esquerda)
    if direita:
        vetor.extend(direita)

    return vetor
```

Algoritmos de Ordenação

Simples/Ineficientes

- **Insertion Sort**
- **Bubble Sort**
- Selection Sort
- ...

Eficientes

- Heap Sort
- **Quick Sort**
- **Merge Sort**
- Tim Sort
- Counting Sort
- ...

Módulos / Bibliotecas

Na prática, usamos as funcionalidades disponíveis.

sorted(*iterable*, /, *, *key=None*, *reverse=False*)

Retorna uma nova lista classificada dos itens em *iterable*.

Possui dois argumentos opcionais que devem ser especificados como argumentos nomeados.

key especifica a função de um argumento usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). O valor padrão é `None` (compara os elementos diretamente).

reverse é um valor booleano. Se definido igual a `True`, então os elementos da lista são classificados como se cada comparação estivesse invertida.

Módulos / Bibliotecas

sorted(*iterable*, /, *, *key=None*, *reverse=False*)

Retorna uma nova lista classificada dos itens em *iterable*.

Possui dois argumentos opcionais que devem ser especificados como argumentos nomeados.

key especifica a função de um argumento usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). O valor padrão é `None` (compara os elementos diretamente).

reverse é um valor booleano. Se definido igual a `True`, então os elementos da lista são classificados como se cada comparação estivesse invertida.

Ordenação em Python



Python HOWTOs » HowTo - Ordenação
https://docs.python.org/pt-br/3/howto/sorting.html?highlight=sort

Resumo

- Ordenação pode ser feita de diferentes formas.
- Alguns algoritmos são mais eficientes do que outros.
- Normalmente algoritmos de divisão e conquista são mais eficientes.
- Python implementa algoritmos eficientes de ordenação.